

LCR 007. 三数之和

中等 相关企业 已解答

给定一个包含 n 个整数的数组 `nums`，判断 `nums` 中是否存在三个元素 a 、 b 、 c ，使得 $a + b + c = 0$ ？请找出所有和为 0 且不重复的三元组。

示例 1:

输入: `nums = [-1,0,1,2,-1,-4]`
输出: `[[-1,-1,2],[-1,0,1]]`

示例 2:

输入: `nums = []`
输出: `[]`

示例 3:

输入: `nums = [0]`
输出: `[]`

提示:

- $0 \leq \text{nums.length} \leq 3000$
- $-10^5 \leq \text{nums}[i] \leq 10^5$

注意: 本题与主站 15 题相同: <https://leetcode.cn/problems/3sum/>

面试中遇到这道题? 1/5

难 简单

通过次数 87,644/202.4K 通过率 43.3%

相关企业

```
1 class Solution {
2 public:
3     vector<vector<int>> threeSum(vector<int>& nums) {
4         // 不返回结果
5         int n = nums.size();
6         // 将数组排序
7         sort(nums.begin(), nums.end());
8         // 结果集
9         vector<vector<int>> ans;
10        // 枚举 a
11        for (int first = 0; first < n; ++first) {
12            // 避免重复
13            // a 重复
14            if (first > 0 && nums[first] == nums[first - 1]) {
15                continue;
16            }
17
18            // c 对应的指针初始指向数组的最右端
19            int third = n - 1;
20            // 需要目标值
21            int target = -nums[first];
22            // 枚举 b
23            // 枚举 a 的下一个开始枚举
24            for (int second = first + 1; second < n; ++second) {
25                // 避免和上一次枚举的数不相等
26                // b 重复 如果和前一个数相等则跳过本次循环
27                if (second > first + 1 && nums[second] == nums[second - 1]) {
28                    continue;
29                }
30
31                // 需要保证 b 的指针在 c 的指针的左侧 如果大于目标值 则 right--
32                while (second < third && nums[second] + nums[third] > target) {
33                    --third;
34                }
35                // 如果指针重合，说明 b 已经遍历完
36                // 就不会再满足 a+b+c=0 并且 b < c 了，可以退出循环
37                if (second == third) {
38                    break;
39                }
40                if (nums[second] + nums[third] == target) {
41                    ans.push_back({nums[first], nums[second], nums[third]});
42                }
43            }
44        }
45        return ans;
46    }
```

75. 颜色分类

中等 相关企业 已解答

给定一个包含红色、白色和蓝色，共 n 个元素的数组 `nums`，原地 对它们进行排序，使得相同颜色的元素相邻，并按颜色顺序排列。

我们使用整数 0、1 和 2 分别表示红色、白色和蓝色。

必须在不使用库内置的 `sort` 函数的情况下解决这个问题。

示例 1:

输入: `nums = [2,0,2,1,1,0]`

输出: `[0,0,1,1,2,2]`

解释:

该数组包含两个 0、两个 1 和两个 2。将它们原地排序后，所有 0 排在最前面，接着是所有 1，最后是所有 2。

示例 2:

输入: `nums = [2,0,1]`

输出: `[0,1,2]`

解释:

数组中有且仅有一个 0、一个 1 和一个 2，按 0、1、2 的顺序原地排列。

已解答

```
1 class Solution {
2 public:
3     void sortColors(vector<int>& nums) {
4         // 计数数组
5         vector<int> count(3);
6         // 计数
7         for (auto num: nums) {
8             count[num]++;
9         }
10        // 按原值顺序还原数组
11        int j = 0;
12        for (int i = 0; i < nums.size(); ++i) {
13            while (count[i] != 0) {
14                nums[i] = i;
15                count[i]--;
16                i++;
17            }
18        }
19    }
20 };
```

147. 对链表进行插入排序

中等 相关企业 已解答

给定单个链表的头 `head`，使用 插入排序 对链表进行排序，并返回 排序后链表的头。

插入排序 算法的步骤:

- 插入排序是迭代的，每次只移动一个元素，直到所有元素可以形成一个有序的输出列表。
- 每次迭代中，插入排序只从输入数据中移除一个待排序的元素，找到它在序列中适当的位置，并将其插入。
- 重复直到所有输入数据插入完为止。

下面是插入排序算法的一个图形示例。部分排序的列表(黑色)最初只包含列表中的第一个元素。每次迭代时，从输入数据中删除一个元素(红色)，并就地插入已排序的列表中。

对链表进行插入排序。



示例 1:



已解答

```
1 class Solution {
2 public:
3     ListNode* insertionSortList(ListNode* head) {
4         // 如果为空链表的话直接返回
5         if (head == nullptr) {
6             return head;
7         }
8         // 哨兵头节点初始值为0
9         ListNode* dummyHead = new ListNode(0);
10        dummyHead->next = head;
11        // 下一个要排序的节点
12        ListNode* lastSorted = head;
13        // 当前节点 用于遍历原链表
14        ListNode* curr = head->next;
15        while (curr != nullptr) {
16            if (lastSorted->val <= curr->val) {
17                lastSorted = lastSorted->next;
18            } else {
19                // 1. prev 指向合理插入位置
20                ListNode* prev = dummyHead;
21                while (prev->next->val <= curr->val) {
22                    prev = prev->next;
23                }
24                lastSorted->next = curr->next;
25                curr->next = prev->next;
26                prev->next = curr;
27            }
28            curr = lastSorted->next;
29        }
30        return dummyHead->next;
31    }
32 };
```